Universidad Nacional Autónoma de México

Matemáticas Aplicadas y Computación

Temas selectos de Computación II

Envió de Correo Electrónico con Spring Boot

Profesor

Padilla Alemán David Bryan

Alumno

Hernández Espinoza Gabriel Emanuel

Usando SpringBoot Initializr agregue

Java Mail Sender: Se usa para **enviar correos electrónicos desde una aplicación Java**, especialmente en proyectos con **Spring Boot** como correos de bienvenida, facturas,notificaciones,etc.

Thymeleaf: Se usa para mostrar en una página web la información que tengo en mi programa Java.

SpringWeb: Spring Web sirve para que la aplicación Java pueda recibir y responder peticiones web, es decir, permite que el proyecto hable con el navegador a través de una URL

En mi proyecto me fui a src/main/resources/templates donde cree un archivo html llamado email.template.html donde puse el siguiente código el cual, su función es generar el cuerpo del correo electrónico que se enviará.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<!--
La parte xmlns:th sirve para activar Thymeleaf.
Le estamos diciendo al archivo que usaremos atributos especiales como:
th:text, th:each, th:if, etc.
Sin esto, Thymeleaf no podría reconocer esas instrucciones.
-->

<head>
   <meta charset="UTF-8">
   <title>Bienvenido</title>
</head>

<body>
   <h1 th:text="'¡Hola, ' + ${nombre} + '!'">¡Hola!</h1>
   <!--
   th:text es de Thymeleaf y sirve para insertar texto dinámico.
   -->

   <p>Este es un correo de confirmacion.</p>

   <p>Este seria un codigo de seguridad:
       <strong th:text="${codigo}">0000</strong>
   </p>
```

```
    <!--

    th:text="${codigo}" toma el valor de la variable codigo
    enviada desde Java.

    Por ejemplo, si desde Java mandas 4821,
    aquí aparecerá: 4821

    El 0000 es solo texto de respaldo.
    -->

    <br>
    <p>Prueba de correo EXITOSA.</p>
</body>

</html>
```

Posterior a esto, en java/com/correo/demo, cree dos carpetas, la primera se llama “service” y la otra “controller”

Dentro de service cree un .java llamado EmailService, con el siguiente código:

```
package com.correo.demo.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;
import org.thymeleaf.TemplateEngine;
import org.thymeleaf.context.Context;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;


@Service //Indica que todo lo siguiente es un Servicio, en este caso, el
de enviar un correo
// Le dice a Spring que esta clase es un servicio para usarla en otras
partes.
```

```java
public class EmailService {

   @Autowired //Inyect un objeto dentro de otro
   // Inyecta automáticamente el objeto encargado de enviar correos.
   private JavaMailSender mailSender;

   @Autowired
   // Inyecta el motor que procesa las plantillas HTML.
   private TemplateEngine templateEngine;

   public void enviarCorreoTemplate(String destinatario, String asunto,
           String nombreUsuario, String codigo)
           throws MessagingException {
       // Método que envía un correo usando plantilla HTML.

       MimeMessage message = mailSender.createMimeMessage();
       // Crea el correo como objeto.

       MimeMessageHelper helper = new MimeMessageHelper(message, true,
"UTF-8");
       // Ayuda a configurar el correo.
       // true = permite HTML y multipart
       // UTF-8 = soporta acentos y caracteres especiales

       Context context = new Context();
       // Crea el contexto para mandar datos al HTML.

       context.setVariable("nombre", nombreUsuario);
       context.setVariable("codigo", codigo);

       String contenidoHtml = templateEngine.process("email-template",
context);
       // Procesa la plantilla HTML llamada email-template.html
       // y reemplaza las variables con sus valores.

       helper.setTo(destinatario);
       helper.setSubject(asunto);
       helper.setText(contenidoHtml, true);
       // Inserta el contenido HTML en el correo.
       // true = indica que es HTML.
```

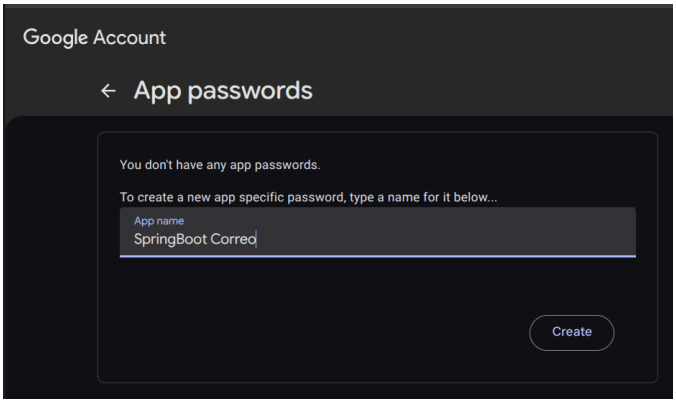
```
        mailSender.send(message);
    }
}
```

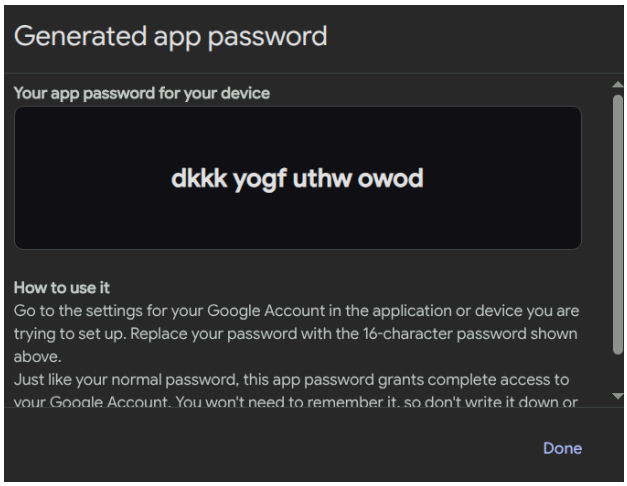
Mientras que en controller cree un .java llamado EmailController con el siguiente código:

```
package com.correo.demo.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.correo.demo.service.EmailService;

import jakarta.mail.MessagingException;
// Importa la excepción por errores al enviar correos.

@RestController
// Le dice a Spring que esta clase controlará peticiones web.
public class EmailController {

    @Autowired //Autowired es para pasar un objeto a otro objeto
    // Inyecta automáticamente el servicio de correos.
    private EmailService emailService;

    @GetMapping("/enviar-correo")
    // Indica que cuando entren a esta URL: /enviar-correo, se ejecutará el
siguiente método.
    public String enviar(@RequestParam String email,
            @RequestParam String nombre) {
        // Método que recibe email y nombre desde la URL.

        try {
            // Intenta ejecutar el envío del correo.

            String codigo = "ABC-123";
            // Simula un código de bienvenida o seguridad.
```

```
            emailService.enviarCorreoTemplate(
                    email,
                    "Correo enviado con SpringBoot",
                    nombre,
                    codigo
            );
            return "Correo enviado con éxito a: " + email;
        } catch (MessagingException e) {
            return "Error al enviar el correo: " + e.getMessage();
        }
    }
}
```

Posterior a esto, en mi cuenta de google (yo ya tenia activada la doble autenticacion) tuve que crear una “Contraseña de Aplicacion”

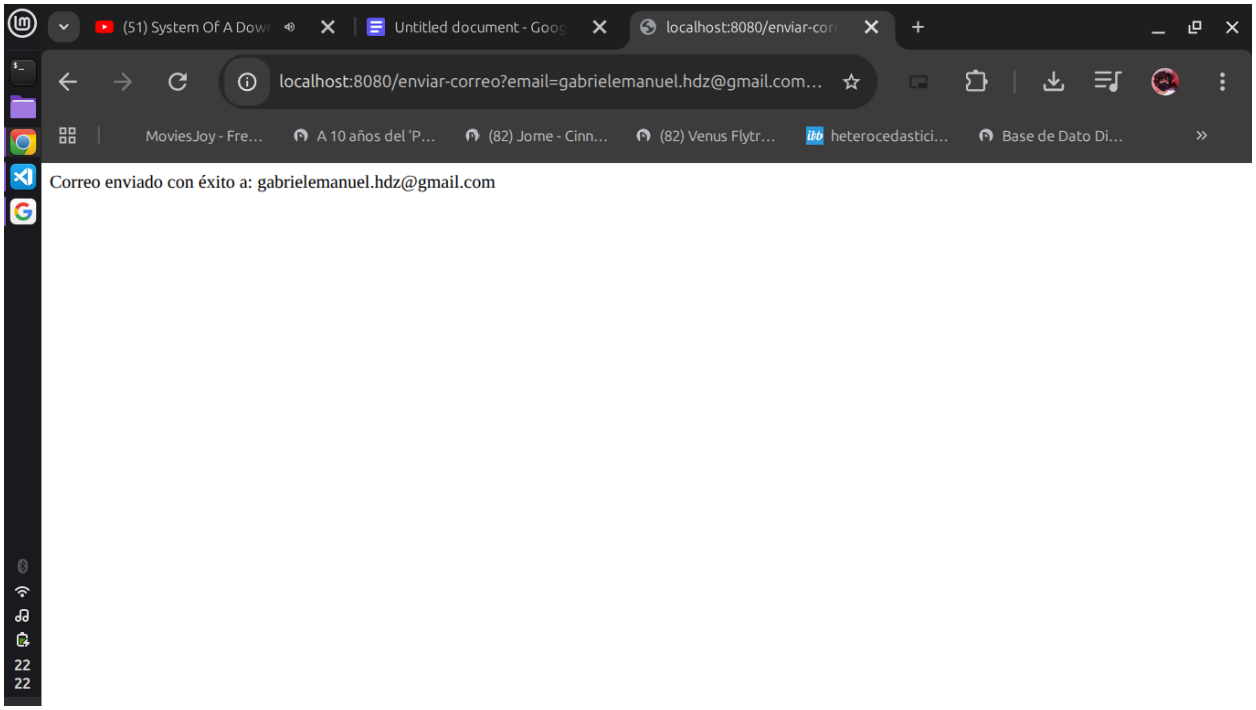


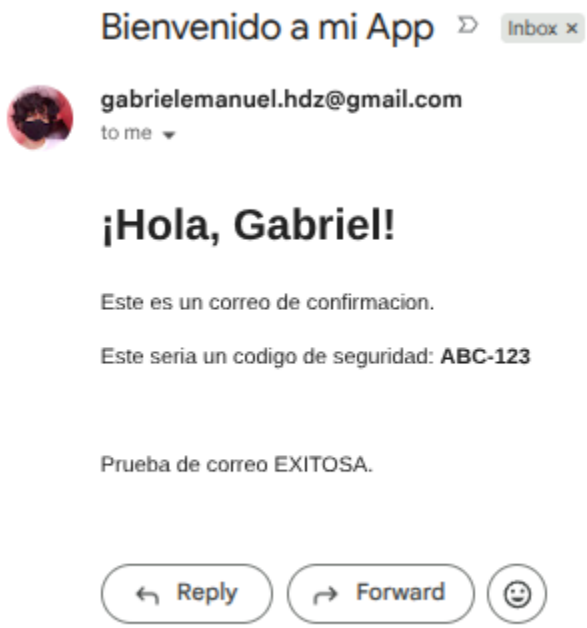


Con este código que google me dio, se hizo lo siguiente en src/main/aplication.propierties, el cual sirve para que el proyecto Spring sepa cómo comunicarse con Gmail

```
spring.application.name=demo
# Define el nombre de la aplicación Spring Boot.
# Sirve para identificar el proyecto internamente.

# Datos de conexión
spring.mail.host=smtp.gmail.com
# Define el servidor SMTP que se usará para enviar correos.
# En este caso se usa el servidor de Gmail.

spring.mail.port=587
# Define el puerto de conexión.
# El 587 se usa para SMTP con cifrado TLS.

spring.mail.username=gabrielemanuel.hdz@gmail.com

# Aqui van las 16 letras que google me proporciono, deben ir sin espacios
spring.mail.password=dkkkyogfuthwowod
# Permite que Java envie correos

# Protocolos de seguridad
spring.mail.properties.mail.smtp.auth=true
# Activa la autenticación.
# Le dice al servidor que se enviará usuario y contraseña.

spring.mail.properties.mail.smtp.starttls.enable=true
# Activa el cifrado TLS.
# Sirve para que la conexión sea segura y protegida.
```

Por ultimo con el link
http://localhost:8080/enviar-correo?email=gabrielemanuel.hdz@gmail.com&nombre=Gabriel


(51) System Of A Dow
Untitled document - Goog
localhost:8080/enviar-cor
localhost:8080/enviar-correo?email=gabrielemanuel.hdz@gmail.com...
MoviesJoy - Fre...
A 10 años del 'P...
(82) Jome - Cinn...
(82) Venus Flytr...
heterocedastici...
Base de Dato Di...
Correo enviado con éxito a: gabrielemanuel.hdz@gmail.com


Bienvenido a mi App
Inbox
gabrielemanuel.hdz@gmail.com
to me
¡Hola, Gabriel!
Este es un correo de confirmacion.
Este seria un codigo de seguridad: ABC-123
Prueba de correo EXITOSA.
Reply
Forward